BLOQUE III - TEMA 4

DESARROLLO DE SISTEMAS

Diseño y programación orientada a objetos. Elementos y componentes software: objetos, clases, herencia, métodos, sobrecarga. Ventajas e inconvenientes. Patrones de diseño y lenguaje de modelado unificado (UML).

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. INTRODUCCIÓN AL PARADIGMA ORIENTADO A OBJETOS 5

1.1. Concepto y evolución 5

1.2. Comparación programación estructurada vs POO 5

1.3. Principios fundamentales 6

2. ELEMENTOS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS 7

2.1. Objetos y clases 7

2.2. Atributos y métodos 7

2.3. Constructores y destructores 8

2.4. Visibilidad y control de acceso 9

2.5. Principios SOLID 10

2.6. Patrones GRASP (General Responsibility Assignment Software Patterns) 11

3. MECANISMOS DE LA ORIENTACIÓN A OBJETOS 12

3.1. Herencia 12

3.2. Polimorfismo 15

3.3. Sobrecarga de métodos 15

3.4. Composición y agregación 16

4. PATRONES DE DISEÑO (DESIGN PATTERNS) 18

4.1. Patrones CREACIONALES 19

4.2. Patrones ESTRUCTURALES 19

4.3. Patrones de COMPORTAMIENTO 20

4.4. Patrones arquitectónicos: MVC 20

5. LENGUAJE DE PROGRAMACIÓN ORIENTADOS A OBJETOS ... 21

5.1. Lenguajes clásicos: Smalltalk, C++ 21

5.2. Lenguajes actuales más usados: Java, Python, C#, Kotlin 21

5.3. Java 22

5.3.1. Estructura del lenguaje 22

5.3.2. Java y la orientación a objetos 22

5.3.3. Ecosistema y aplicaciones 23

6. MODELADO ORIENTADO A OBJETOS CON UML 24

6.1. Diagramas estructurales 24

6.1.1. Diagrama de clases 24

6.1.2. Diagrama de objetos 25

6.1.3. Diagrama de paquetes 26

6.1.4. Diagrama de componentes 26

6.1.5. Diagrama de estructura 27

6.1.6. Diagrama de implementación (o despliegue) 27

6.2. Diagramas de comportamiento (secuencia, casos de uso, actividad) 28

6.2.1.Diagrama de casos de uso 28

6.2.2. Diagramas de interacción 29

6.2.3. Diagrama de actividad 31

6.2.4. Diagrama de estados (máquina de estados) 31

6.2.5. Diagrama de tiempo (temporal) 31

6.2.6. Diagramas de flujo de datos (DFD) 31

7. FLUJOGRAMAS O DIAGRAMAS DE FLUJO DE PROCESOS 37

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. INTRODUCCIÓN AL PARADIGMA ORIENTADO A OBJETOS

1.1. CONCEPTO Y EVOLUCIÓN

La **programación orientada a objetos (POO)** es un paradigma de programación basado en el concepto de "objetos", que son entidades que agrupan datos (atributos) y comportamiento (métodos).

- **Años 60-70:** surgen los primeros lenguajes estructurados como C
- **1980:** nace el lenguaje **Smalltalk**, primer lenguaje puramente orientado a objetos.
- **Década de 1980:** se introduce **C++**, que añade orientación a objetos sobre C.
- **Década de 1990:** se consolida con lenguajes como **Java.**
- **Actualidad:** la POO está presente en múltiples lenguajes modernos (Java, C++, C#, Python, Swift, etc.), especialmente en desarrollo de aplicaciones complejas, juegos, interfaces gráficas y software empresarial.

1.2. COMPARACIÓN PROGRAMACIÓN ESTRUCTURADA VS POO

A lo largo de la evolución del desarrollo de software, han surgido diversos paradigmas de programación que ofrecen diferentes enfoques para resolver problemas computacionales: **programación estructurada** y la **programación orientada a objetos (POO)**.

- La **programación estructurada** se basa en la **descomposición** de un problema en una serie de **procedimientos o funciones**, utilizando estructuras de control como secuencias, decisiones e iteraciones. Este enfoque promueve un flujo de control lógico y claro, facilitando la comprensión y el mantenimiento de programas sencillos o de tamaño moderado.
- La **programación orientada a objetos** introduce el concepto de "objetos" que **encapsulan tanto datos como comportamiento**s. Este paradigma permite modelar sistemas complejos de manera más natural y modular, promoviendo la reutilización de código y facilitando el mantenimiento y la escalabilidad del software.

Característica	Programación estructurada	Programación orientada a objetos
Unidad básica	Función o procedimiento	Objeto (instancia de clase)
Organización	Código dividido en funciones	Código dividido en clases y objetos
Estructura de datos	Datos y funciones separados	Datos y funciones agrupados
Reutilización	Baja (requiere copia/adaptación)	Alta (mediante herencia y composición)
Mantenimiento	Difícil en proyectos grandes	Más sencillo al estar modularizado
Abstracción	Procedimientos y estructuras	Clases y objetos
Ejemplo típico de lenguaje	C, Pascal	Java, C++, Python

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1.3. PRINCIPIOS FUNDAMENTALES

EXAMEN

La POO se basa en varios principios clave que permiten un diseño más limpio, mantenible y escalable:

a) Encapsulamiento

- Es la **capacidad de ocultar los detalles internos de un objeto**, exponiendo solo lo necesario.
- Se consigue mediante modificadores de acceso (privado, público, protegido).
- Permite **proteger los datos** y mantener la **integridad** del estado del **objeto**.

b) Abstracción

- Consiste en **representar entidades del mundo real mediante modelos simplificados** (clases).
- Se centra en los **aspectos esenciales**, ignorando los detalles irrelevantes.

c) Modularidad

- El sistema **se divide en módulos (clases) independientes y cohesivos.**
- Cada módulo se encarga de una funcionalidad concreta.
- Favorece el desarrollo colaborativo y la localización de errores.

d) Reutilización

- Se refiere a la capacidad de usar componentes ya existentes en nuevos programas.
- Se logra mediante mecanismos como la herencia, la composición y las bibliotecas de clases.
- Reduce tiempos de desarrollo y mejora la fiabilidad del software.



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. ELEMENTOS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

La programación orientada a objetos se construye a partir de una serie de componentes esenciales que permiten organizar el software en bloques autónomos y reutilizables. Estos elementos definen tanto la estructura de los objetos como su comportamiento, y son la base para cualquier aplicación desarrollada bajo este paradigma.

2.1. OBJETOS Y CLASES

Uno de los pilares fundamentales de la POO es la relación entre clases y objetos. Las clases actúan como moldes o plantillas a partir de las cuales se crean los objetos, que son las unidades reales que se manipulan durante la ejecución del programa.

- **Clase:** Es una plantilla o modelo que define las características (atributos) y comportamientos (métodos) de un conjunto de objetos.
- **Objeto:** Es una instancia de una clase. Representa una entidad concreta que tiene un estado (los valores de sus atributos) y un comportamiento (sus métodos).

Ejemplo en Java:

```
class Persona {
    String nombre;
    int edad;
    void saludar() {
        System.out.println("Hola, me llamo " + nombre);
    }
}
Persona p = new Persona();  // p es un objeto de tipo Persona
```

2.2. ATRIBUTOS Y MÉTODOS

Dentro de cada objeto, encontramos dos elementos esenciales: los atributos y los métodos. Los atributos almacenan información sobre el objeto, mientras que los métodos definen qué puede hacer dicho objeto. Esta división permite encapsular tanto el estado como el comportamiento.

- **Atributos:** Variables que definen el estado del objeto. También se conocen como campos o propiedades.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Métodos:** Funciones que definen el comportamiento del objeto, es decir, las acciones que puede realizar.

Ejemplo:

```
class Coche {
    String color;
    int velocidad;
    void acelerar(int incremento) {
        velocidad += incremento;
    }
}
```

2.3. CONSTRUCTORES Y DESTRUCTORES

Para gestionar adecuadamente el ciclo de vida de los objetos, los lenguajes orientados a objetos incorporan mecanismos especiales de inicialización y liberación de recursos: los constructores y destructores. Su uso adecuado garantiza que los objetos se creen y destruyan de forma controlada.

- **Constructor:** Método especial que se ejecuta automáticamente al crear un objeto. Se utiliza para inicializar atributos.
- **Destructor:** Método que se ejecuta cuando un objeto deja de existir. En algunos lenguajes (como C++) se usa para liberar recursos. En otros, como Java, esta tarea la gestiona el recolector de basura.

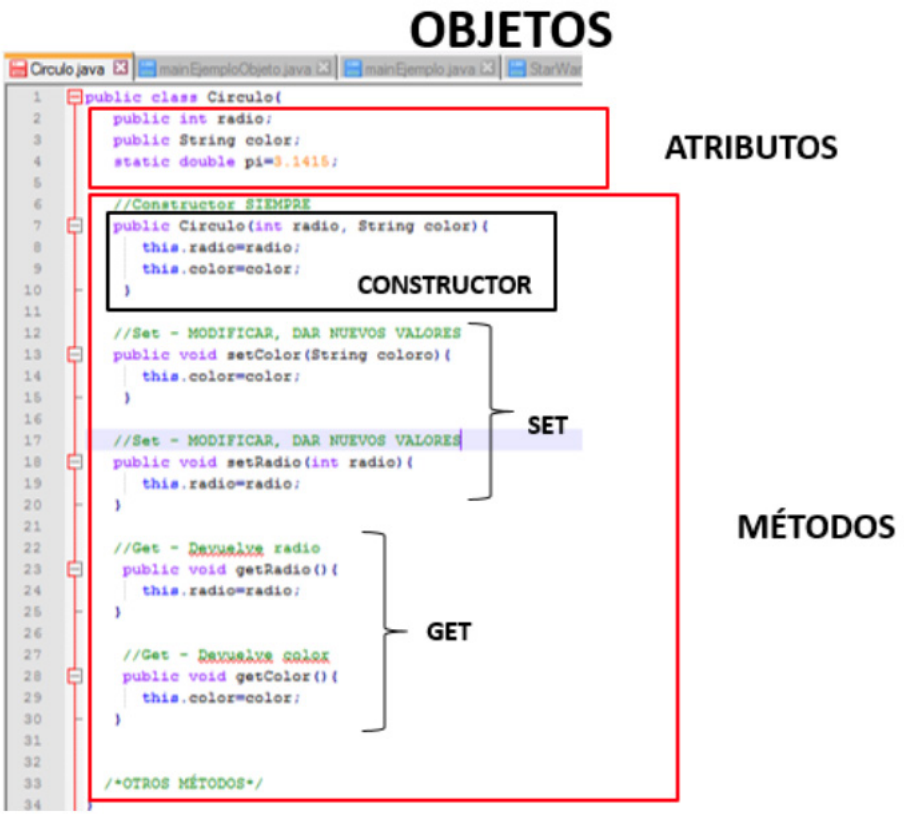
Ejemplo en Java:

```
class Libro {
    String titulo;
    Libro(String t) {
        titulo = t;
    }
}
```

Ejemplo en C++:

```
class Libro {
public:
    Libro() { /* Constructor */ }
    ~Libro() { /* Destructor */ }
};
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



2.4. VISIBILIDAD Y CONTROL DE ACCESO

Uno de los principios clave de la POO es el **encapsulamiento**, que **permite ocultar los detalles internos** de una clase y exponer únicamente lo necesario. Para ello se utilizan modificadores de acceso, que controlan qué partes del código pueden interactuar con los atributos y métodos de una clase.

- Los **modificadores** más comunes son:

Modificador	Desde la clase	Desde subclases	Desde otras clases
private	✅	❌	❌
protected	✅	✅	❌
public	✅	✅	✅
(sin especificar)	✅	❌	❌

- **Sin especificar** (conocido como **default** o **package-private** en Java): si no se indica ningún modificador, el miembro es accesible **únicamente** desde las clases que residen dentro del **mismo paquete** lógico. Se bloquea el acceso a las subclases de otros paquetes, a diferencia de protected.

- Se recomienda mantener los atributos como private y proporcionar acceso mediante métodos públicos llamados *getters* y *setters*.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo:

```
class Usuario {
    private String password;
    public void setPassword(String p) {
        password = p;
    }
    public String getPassword() {
        return password;
    }
}
```

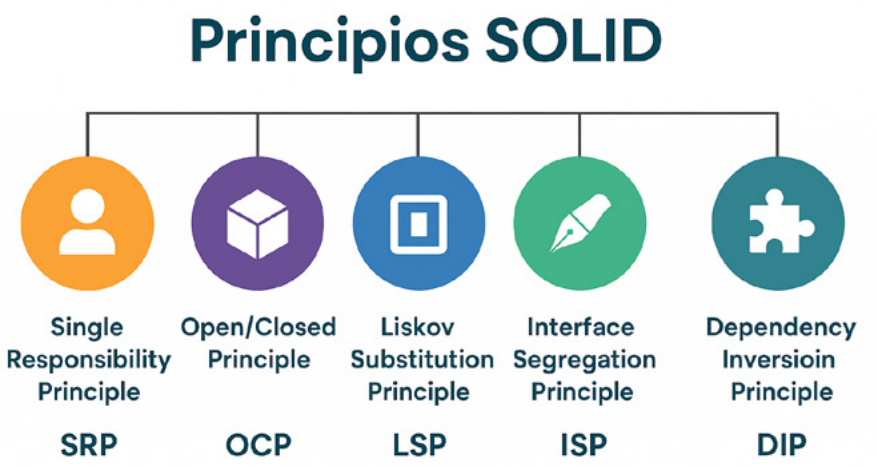
2.5. PRINCIPIOS SOLID



Los principios **SOLID** resumen buenas prácticas para el diseño orientado a objetos. Son:

1. **S - Single Responsibility Principle (SRP)**
   Cada clase debe tener una única responsabilidad o razón para cambiar.
2. **O - Open/Closed Principle (OCP)**
   El software debe estar **abierto a la extensión pero cerrado a la modificación**.
3. **L - Liskov Substitution Principle (LSP)**
   Las clases derivadas deben poder sustituir a sus clases base **sin alterar el funcionamiento** del programa.
4. **I - Interface Segregation Principle (ISP)**
   Es mejor tener **muchas interfaces específicas** que una sola general que obligue a implementar métodos innecesarios.
5. **D - Dependency Inversion Principle (DIP)**
   Los módulos de alto nivel no deben depender de módulos de bajo nivel, **sino de abstracciones**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



2.6. PATRONES GRASP (GENERAL RESPONSIBILITY ASSIGNMENT SOFTWARE PATTERNS)

EXAMEN

GRASP es un conjunto de **patrones orientados a objetos** que ayudan a **asignar responsabilidades a clases y objetos** dentro del diseño de software. Su objetivo es facilitar la toma de decisiones en el diseño orientado a objetos, promoviendo soluciones coherentes, reutilizables y mantenibles.

Los patrones GRASP no son patrones de diseño como *Factory* o *Singleton*, sino principios generales para guiar la asignación de responsabilidades. Fueron introducidos por **Craig Larman** y están muy relacionados con los conceptos de **cohesión, acoplamiento, encapsulamiento y abstracción**.

Principales patrones GRASP

1. **Información experta (Information Expert):** Asigna una responsabilidad a la clase que tiene la mayor parte de la información necesaria para cumplirla.
2. **Creador (Creator):** Establece qué clase debe ser responsable de crear instancias de otra clase.
3. **Controlador (Controller):** Introduce una clase intermediaria que recibe eventos del sistema (como acciones del usuario) y coordina las respuestas apropiadas.
4. **Alta cohesión (High Cohesion):** Favorece que cada clase tenga un conjunto claro y estrechamente relacionado de responsabilidades.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. **Bajo acoplamiento (Low Coupling):** Reduce la dependencia entre clases para facilitar cambios y reutilización.
6. **Polimorfismo (Polymorphism):** Usa el polimorfismo para manejar variantes en el comportamiento según el tipo.
7. **Fabrica pura (Pure Fabrication):** Introduce una clase artificial para mantener bajo acoplamiento y alta cohesión, cuando no encaja con una entidad del dominio.
8. **Indirección (Indirection):** Introduce un objeto intermediario para reducir el acoplamiento entre clases.
9. **Variación protegida (Protected Variations):** Protege partes del sistema de los efectos de posibles cambios introduciendo interfaces estables.

3. MECANISMOS DE LA ORIENTACIÓN A OBJETOS

Los mecanismos que proporciona la programación orientada a objetos permiten construir sistemas más flexibles, reutilizables y adaptables. Gracias a ellos, se puede establecer relaciones entre clases, redefinir comportamientos o combinar objetos para crear estructuras más complejas. Los más importantes son la herencia, el polimorfismo, la sobrecarga y la composición.

3.1. HERENCIA

La **herencia** permite que una clase (subclase o clase derivada) **herede atributos y métodos de otra** (superclase o clase base). Este mecanismo favorece la reutilización de código, la extensión de funcionalidades y la creación de jerarquías lógicas entre clases.

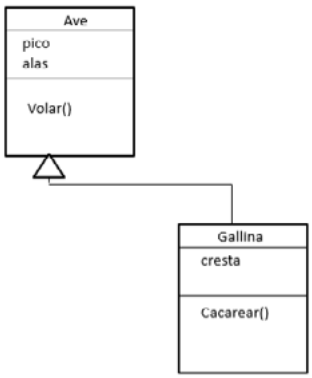
- Permite crear una nueva clase basándose en una ya existente.
- La subclase puede sobrescribir (override) métodos de la superclase.
- Facilita la reutilización y el mantenimiento del código.
- La subclase hereda automáticamente los atributos, métodos y relaciones de la superclase. Además, puede **añadir** o **especializar** su propio comportamiento o propiedades.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

EXAMEN

Ejemplo en Java:

```
class Ave {
    void volar() {
        System.out.println("Vuela como un ave");
    }
}
class Gallina extends Ave {
    void volar() {
        System.out.println("No vuela");
    }
    void cacarear(){
        System.out.println("Co co co");
}
```



Superclase: En programación orientada a objetos (POO), una **superclase** es una **clase general o base** de la que pueden derivar otras clases más especializadas, llamadas **subclases**. La superclase contiene atributos y métodos comunes que serán **heredados** por sus subclases, promoviendo la reutilización de código y la jerarquía conceptual.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Características principales de una superclase:**
  - Define **comportamiento y estructura comunes** a varias clases.
  - Puede incluir métodos que se heredan tal cual o que pueden ser **sobrescritos** en las subclases.
  - Es la base de la relación de **herencia**, que permite crear jerarquías de clases.
  - En muchos lenguajes, una subclase solo puede heredar de **una única superclase** (herencia simple), aunque algunos permiten **herencia múltiple**.
- **Ejemplo en Java:**

```
public class Animal {
    public void hacerSonido() {
        System.out.println("Algún sonido");
    }
}
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

Herencia simple: Una clase solo puede derivar de una única superclase. Esto simplifica la jerarquía, evita ambigüedades y reduce la complejidad del modelo de objetos.

- **Java**: Una clase solo puede extender a otra (extends), pero puede implementar múltiples interfaces.
- **C#**: Permite herencia simple de clases y herencia múltiple de interfaces (interface).
- **Swift**: permite herencia simple de clases (class), pero herencia múltiple de protocolos (equivalentes a interfaces).
- **Kotlin**: herencia simple, pero permite heredar múltiples interfaces.
- **Objective-C**: solo herencia simple. Admite protocolos múltiples como interfaces.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Herencia múltiple: La herencia múltiple permite que una clase herede de más de una superclase. Esto ofrece mayor expresividad, pero puede generar problemas de ambigüedad.

- **C++:** Usa mecanismos como la herencia virtual para resolver ambigüedades.
- **Python:** Usa el algoritmo MRO para definir el orden de resolución.
- **Ruby:** no tiene herencia múltiple directa, pero usa mixins (módulos) para incorporar funcionalidades de múltiples fuentes.

3.2. POLIMORFISMO



El **polimorfismo** permite que una **misma operación tenga diferentes comportamientos** según el tipo de objeto que la utilice. Esto se traduce en que **distintas clases pueden compartir el mismo nombre de método pero con implementaciones distintas**. Permite tratar objetos de diferentes clases como si fueran del mismo tipo base.

- **Estático (en tiempo de compilación):** mediante sobrecarga.
- **Dinámico (en tiempo de ejecución):** mediante herencia y sobreescritura.

Ejemplo en Java:

```
Animal miAnimal = new Perro();

miAnimal.sonido();  // Llama al método de Perro, no de Animal
```

3.3. SOBRECARGA DE MÉTODOS



La **sobrecarga** es un mecanismo que permite definir múltiples versiones de un método dentro de una misma clase, **diferenciándolos por el número o tipo de parámetros.**

- **Sobrecarga de métodos:** varios métodos con el mismo nombre pero distinta forma.
- **Sobrecarga de operadores:** se redefine el comportamiento de operadores estándar (disponible en C++).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo de sobrecarga de métodos en Java:

```
class Calculadora {
    int sumar(int a, int b) {
        return a + b;
    }
    double sumar(double a, double b) {
        return a + b;
    }
}
```

Ejemplo de sobrecarga de operadores en C++:

```
class Complejo {
public:
    float real, imag;
    Complejo operator + (const Complejo& c) {
        return {real + c.real, imag + c.imag};
    }
};
```

Sobrecarga y sobrescritura

- **Sobrecarga (Overloading):** varios métodos de la **misma clase** comparten nombre, pero tienen **distintos parámetros** (número, tipo u orden). Se resuelve **estáticamente**, en tiempo de compilación.
- **Sobrescritura (Overriding):** se produce cuando una **subclase** da una implementación específica a un método ya definido en su **superclase**. Requiere la **misma firma** (nombre y parámetros). Se resuelve **dinámicamente**, en tiempo de ejecución.

Aspecto	Sobrecarga (Overloading)	Sobrescritura (Overriding)
Ámbito	Misma clase	Superclase y subclase (herencia)
Firma del método	Mismo nombre, distintos parámetros	Mismo nombre y mismos parámetros
Resolución	Estática (en compilación)	Dinámica (en ejecución)
Tipo de polimorfismo	Estático	Dinámico

3.4. COMPOSICIÓN Y AGREGACIÓN

Composición y **agregación** son formas de establecer relaciones entre clases que representan una parte-todo (relaciones "tiene un"), pero con distintos grados de dependencia.

- **Composición:** relación fuerte. Si el objeto "todo" se destruye, también sus partes.
- **Agregación:** relación débil. Las partes pueden existir independientemente del todo.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo:

```
class Motor {}  // Clase parte
class Coche {
   private Motor motor;  // Composición: el motor no existe sin el coche
}
class Profesor {}
class Departamento {
    private List<Profesor> profesores;  // Agregación: los profesores
pueden existir sin el departamento
}
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4. PATRONES DE DISEÑO (DESIGN PATTERNS)

Un **patrón de diseño** es una solución general reutilizable para un problema común dentro de un contexto específico en el diseño de software. Los patrones **no son piezas de código listas para copiar, sino modelos que proponen buenas prácticas** para resolver problemas habituales en el diseño orientado a objetos.

GoF (Gang of Four) es el nombre con el que se conoce a los autores del libro **"Design Patterns: Elements of Reusable Object-Oriented Software"**, publicado en 1994 por **Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides**. Esta obra se considera un pilar fundamental en el diseño de software orientado a objetos.

En el libro, los autores describen **23 patrones de diseño clasificados en tres tipos**:

- **Creacionales**: enfocan en la creación de objetos (ej.: Singleton, Factory Method).
- **Estructurales**: describen cómo componer clases y objetos (ej.: Adapter, Decorator, Composite).
- **De comportamiento**: definen cómo interactúan los objetos (ej.: Observer, Strategy, Chain of Responsibility).

Tipo de patrón	Finalidad principal
Creacionales	Gestionan la creación de objetos de forma flexible y controlada
Estructurales	Definen cómo se relacionan las clases y objetos entre sí
De comportamiento	Se centran en la comunicación y colaboración entre objetos

Estos patrones ofrecen **soluciones reutilizables y probadas** para problemas comunes de diseño de software, promoviendo la modularidad, extensibilidad y mantenibilidad del código.

Objetivos principales:

- Fomentar la reutilización del conocimiento de diseño.
- Mejorar la mantenibilidad y extensibilidad del software.
- Facilitar la comunicación entre desarrolladores al usar un vocabulario común.
- Evitar la reinvención de soluciones para problemas ya conocidos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.1. PATRONES CREACIONALES

Los **patrones creacionales** abordan el problema de cómo se crean los objetos en un sistema orientado a objetos. En lugar de instanciar objetos directamente con el operador new, **estos patrones ofrecen soluciones más flexibles, desacoplando el código** cliente de las clases concretas y facilitando la reutilización.

1. **Singleton:** Garantiza que una clase tenga una única instancia y proporciona un acceso global a ella.
2. **Factory Method:** Define una interfaz para crear objetos, delegando a las subclases la decisión de qué clase instanciar.
3. **Abstract Factory:** Permite crear familias de objetos relacionados sin especificar sus clases concretas.
4. **Builder:** Separa la construcción de un objeto complejo de su representación, permitiendo crear distintas representaciones.
5. **Prototype:** Permite copiar objetos existentes sin depender de sus clases concretas.

4.2. PATRONES ESTRUCTURALES

Los patrones estructurales se centran en **cómo se componen clases y objetos para formar estructuras más grandes**, promoviendo una organización flexible y eficiente del código. Estos patrones facilitan la reutilización y la adaptabilidad al permitir modificar o extender el comportamiento sin alterar el código existente.

1. **Adapter:** Permite que clases con interfaces incompatibles trabajen juntas, actuando como traductor entre ellas.
2. **Decorator:** Añade funcionalidad a un objeto de manera dinámica, sin modificar su clase original.
3. **Composite:** Permite tratar objetos individuales y composiciones de objetos de forma uniforme.
4. **Facade:** Proporciona una interfaz unificada y simplificada para un conjunto de interfaces más complejas.
5. **Proxy:** Proporciona un representante o sustituto de otro objeto para controlar su acceso (control de acceso, carga diferida, etc.).
6. **Bridge:** Separa una abstracción de su implementación para que ambas puedan evolucionar de forma independiente.
7. **Flyweight:** Reduce el uso de memoria compartiendo objetos que son similares entre sí.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.3. PATRONES DE COMPORTAMIENTO

Los patrones de comportamiento se centran en **cómo interactúan los objetos y cómo se distribuyen las responsabilidades entre ellos**, facilitando la comunicación, el control del flujo de ejecución y la asignación dinámica de comportamientos. Estos patrones permiten diseñar sistemas más flexibles y mantenibles.

1. **Observer:** Permite que múltiples objetos se mantengan actualizados cuando el estado de otro cambia.
2. **Strategy:** Permite definir una familia de algoritmos intercambiables, y elegir uno de ellos en tiempo de ejecución.
3. **Command:** Encapsula una solicitud como un objeto, permitiendo parametrizar clientes con distintas operaciones.
4. **State:** Permite que un objeto cambie su comportamiento cuando cambia su estado interno.
5. **Template Method:** Define el esqueleto de un algoritmo, dejando algunos pasos a implementar en subclases.
6. **Mediator:** Centraliza la comunicación entre objetos para reducir el acoplamiento.
7. **Iterator:** Permite recorrer los elementos de una colección sin exponer su representación interna.
8. **Chain of Responsibility:** Permite pasar una petición a lo largo de una cadena de objetos hasta que uno la gestione.
9. **Visitor:** Permite definir nuevas operaciones sobre una estructura de objetos sin modificar las clases sobre las que opera.
10. **Interpreter:** Define una gramática y una forma de interpretar oraciones de ese lenguaje.

4.4. PATRONES ARQUITECTÓNICOS: MVC

Los **patrones arquitectónicos** actúan a un nivel superior al de los patrones de diseño: no resuelven problemas concretos entre clases u objetos, sino que definen la **estructura global de una aplicación** y la forma en que se organizan y comunican sus grandes componentes. Algunos ejemplos son la arquitectura en capas, el modelo cliente-servidor o los microservicios.

El más conocido es el **MVC (Modelo-Vista-Controlador)**, que separa la aplicación en tres componentes:

- **Modelo:** contiene las **reglas de negocio** y gestiona el **acceso a los datos**.
- **Vista:** es la **interfaz de usuario**; presenta la información del modelo.
- **Controlador:** actúa como **intermediario**: procesa las peticiones del usuario y coordina el Modelo y la Vista.

Funcionamiento: el usuario interactúa con la Vista; el Controlador recibe la petición, solicita al Modelo las operaciones necesarias y selecciona la Vista que muestra el resultado. Esta separación de responsabilidades facilita el mantenimiento, la reutilización y el trabajo en paralelo de distintos equipos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. LENGUAJE DE PROGRAMACIÓN ORIENTADOS A OBJETOS

El paradigma de la programación orientada a objetos (POO) comenzó a consolidarse en los años 70 y 80 con la aparición de los primeros lenguajes que lo implementaban de forma explícita. Entre ellos destacan **Smalltalk** y **C++.**

5.1. LENGUAJES CLÁSICOS: SMALLTALK, C++

- **Smalltalk**
  - Lenguaje puramente orientado a objetos: todo es un objeto, incluso los números y los bloques de código.
  - Influyó en muchos lenguajes posteriores (Java, Ruby).
  - Uso más académico que industrial.
- **C++**
  - Derivado de C, pero añade soporte para clases, objetos, herencia, sobrecarga y más.
  - Ofrece control de bajo nivel (punteros, gestión de memoria), lo que lo hace potente pero complejo.
  - Usado ampliamente en sistemas embebidos, videojuegos, motores gráficos y sistemas donde se necesita rendimiento.

5.2. LENGUAJES ACTUALES MÁS USADOS: JAVA, PYTHON, C#, KOTLIN

- **Java**
  - Portabilidad gracias a la Java Virtual Machine (JVM).
  - Amplio uso en aplicaciones empresariales, Android y backend.
- **Python**
  - Lenguaje multiparadigma con fuerte soporte para objetos.
  - Sintaxis clara y simple, ideal para principiantes y prototipado rápido.
  - Muy usado en ciencia de datos, inteligencia artificial y automatización.
- **C#**
  - Desarrollado por Microsoft como alternativa a Java.
  - Sintaxis y enfoque similares, con integración en el ecosistema .NET.
  - Muy empleado en aplicaciones de escritorio, web y videojuegos (con Unity).
- **Kotlin**
  - Diseñado para interoperar completamente con Java.
  - Moderno, conciso, seguro frente a errores como null.
  - Lenguaje oficial para Android desde 2017.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5.3. JAVA

Java es un lenguaje de programación orientado a objetos creado por **James Gosling y su equipo en Sun Microsystems** en 1995. Su lema, *"write once, run anywhere"*, refleja su capacidad de ejecutarse en cualquier plataforma que tenga instalada la **Java Virtual Machine (JVM)**.

Características principales:

- **Orientado a objetos**: Todo gira en torno a clases y objetos, aunque existen tipos primitivos.
- **Multiplataforma**: Gracias a la **JVM**, el mismo bytecode puede ejecutarse en distintos sistemas.
- **Fuertemente tipado y de tipado estático**: Los tipos se definen explícitamente y se comprueban en tiempo de compilación.
- **Con recolección automática de memoria (Garbage Collector)**: Reduce el riesgo de fugas de memoria.
- **Seguro**: La JVM ejecuta el código en una "sandbox" que limita comportamientos maliciosos o peligrosos.
- **Multihilo (multithreading)**: Incorpora soporte para concurrencia desde el núcleo del lenguaje.
- **Extensible y mantenible**: Gracias a una arquitectura modular y un amplio conjunto de bibliotecas estándar.

5.3.1. Estructura del lenguaje

Un programa en Java se organiza en **clases**, dentro de **paquetes**. Cada clase puede contener atributos, métodos y constructores. El punto de entrada a la ejecución es el método main.

Ejemplo básico:

```
public class HolaMundo {

    public static void main(String[] args) {

        System.out.println("¡Hola, mundo!");

    }

}
```

5.3.2. Java y la orientación a objetos

Java implementa de forma rigurosa los principios de la programación orientada a objetos:

- **Abstracción**: Clases abstractas e interfaces permiten definir contratos y estructuras generales.
- **Encapsulamiento**: Mediante modificadores de acceso (private, public, protected), los atributos se ocultan y se accede a ellos mediante métodos.
- **Herencia**: Java admite herencia simple entre clases (extends) y herencia múltiple a través de interfaces (implements).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Polimorfismo**: Permite definir métodos con el mismo nombre pero distinto comportamiento según el tipo de objeto.

5.3.3. Ecosistema y aplicaciones

Java cuenta con un ecosistema vasto, que incluye:

- **Java SE (Standard Edition)**: Para aplicaciones de escritorio y consola.
- **Java EE (Enterprise Edition)**: Para aplicaciones empresariales distribuidas (hoy Jakarta EE).
- **Java ME (Micro Edition)**: Para dispositivos con recursos limitados.
- **Android SDK**: Aunque Android se ha orientado a Kotlin, Java sigue siendo una de las principales opciones para desarrollar aplicaciones móviles.

Entre los frameworks más conocidos se encuentran **Spring**, **Hibernate**, **JSF**, **JUnit** (pruebas), y herramientas como **Maven** o **Gradle** para la gestión de dependencias.



En el tema 5 del Bloque III se realiza un análisis completo de Java

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6. MODELADO ORIENTADO A OBJETOS CON UML

El **Lenguaje de Modelado Unificado (UML)** es el estándar más utilizado para representar visualmente los sistemas orientados a objetos. NO es un lenguaje de programación, sino un lenguaje de modelado que se puede aplicar en cualquier fase del ciclo de vida del software.

UML (**Unified Modeling Language**) fue desarrollado en los años 90 por Grady Booch, Ivar Jacobson y James Rumbaugh, y estandarizado por la OMG (Object Management Group).

6.1. DIAGRAMAS ESTRUCTURALES



Los **diagramas estructurales** en UML describen la **estructura estática** del sistema. Permiten representar qué elementos componen el sistema, cómo se relacionan entre ellos y cómo están organizados. Son fundamentales en el diseño orientado a objetos porque permiten visualizar las dependencias y modularidad del sistema antes de programarlo.

6.1.1. Diagrama de clases

Es el **más representativo** en el contexto de la programación orientada a objetos. Muestra las **clases que forman parte del sistema**, sus **atributos**, **métodos** y las **relaciones** entre ellas: asociación, agregación, composición, dependencia, herencia (generalización), entre otras.

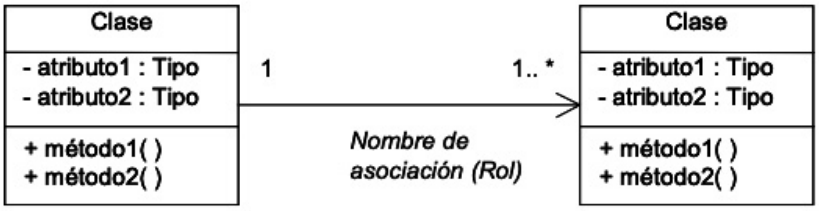
El diagrama de clases recoge las clases de objetos y sus asociaciones. En este diagrama se representa la estructura y el comportamiento de cada uno de los objetos del sistema y sus relaciones con los demás objetos, pero **no muestra información temporal**.

Un diagrama de clase típico representa cada clase en un **rectángulo** dividido en **tres secciones:**

1. **Nombre** de la clase.
2. **Atributos** (visibilidad, nombre y tipo). Los valores de visibilidad son:
   - + público (public)
   - - privado (private)
   - # protegido (protected)
3. **Métodos** u operaciones (visibilidad, nombre, parámetros y tipo de retorno).

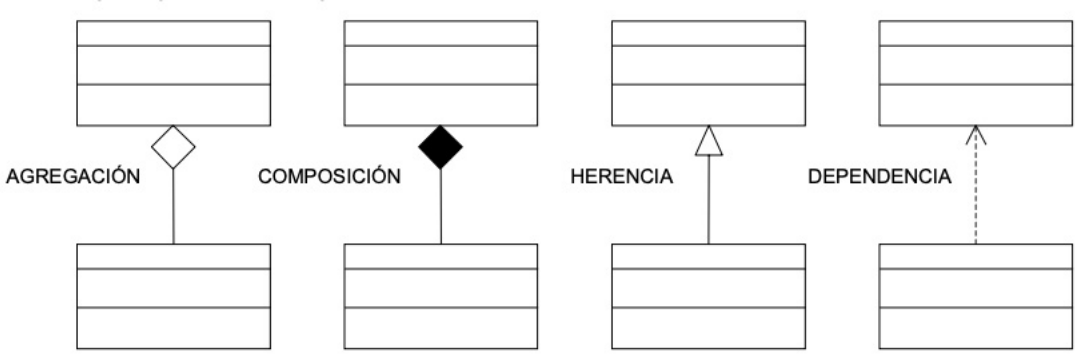
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo básico:



Relaciones comunes:

- **Asociación**: relación entre objetos, como "un Cliente tiene un Pedido".
- **Herencia (generalización)**: una clase hereda de otra.
- **Composición**: una clase contiene a otra de forma exclusiva (por ejemplo, un Coche contiene un Motor).
- **Agregación**: relación de todo/parte sin pertenencia exclusiva (una Universidad agrupa Departamentos).



- **Realización (implementación):** relación entre una interfaz y la clase que la implementa.

Notación gráfica de las relaciones:

Relación	Tipo de línea	Extremo
Asociación	Continua	Sin decoración (una flecha abierta indica navegabilidad)
Herencia (generalización)	Continua	Triángulo hueco, apuntando a la superclase
Realización (implementación)	Discontinua	Triángulo hueco, apuntando a la interfaz
Composición	Continua	Rombo negro (relleno), en el lado del todo
Agregación	Continua	Rombo blanco (hueco), en el lado del todo
Dependencia	Discontinua	Flecha abierta, apuntando al elemento del que se depende

6.1.2. Diagrama de objetos

Este diagrama representa una **vista en tiempo de ejecución** del sistema, mostrando **instancias concretas** de las clases (es decir, objetos) y sus valores actuales.

Usos:

- Visualizar cómo interactúan los objetos en un momento específico.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

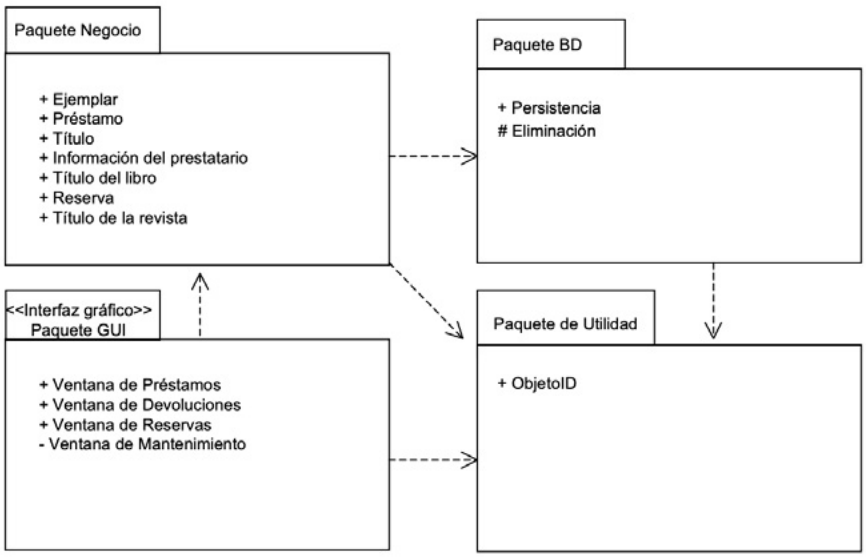
- Ilustrar ejemplos concretos o casos de prueba.
- Entender cómo se instancian y relacionan las clases en la práctica.

6.1.3. Diagrama de paquetes

Permite **agrupar clases relacionadas** en **paquetes** o módulos lógicos. Es útil para organizar el sistema en capas, como presentación, lógica de negocio, acceso a datos, etc.

Cada paquete puede contener otras clases, interfaces o incluso otros paquetes, y las relaciones entre paquetes pueden reflejar **dependencias de importación o acoplamiento**. Estos diagramas contienen **dos tipos de elementos:**

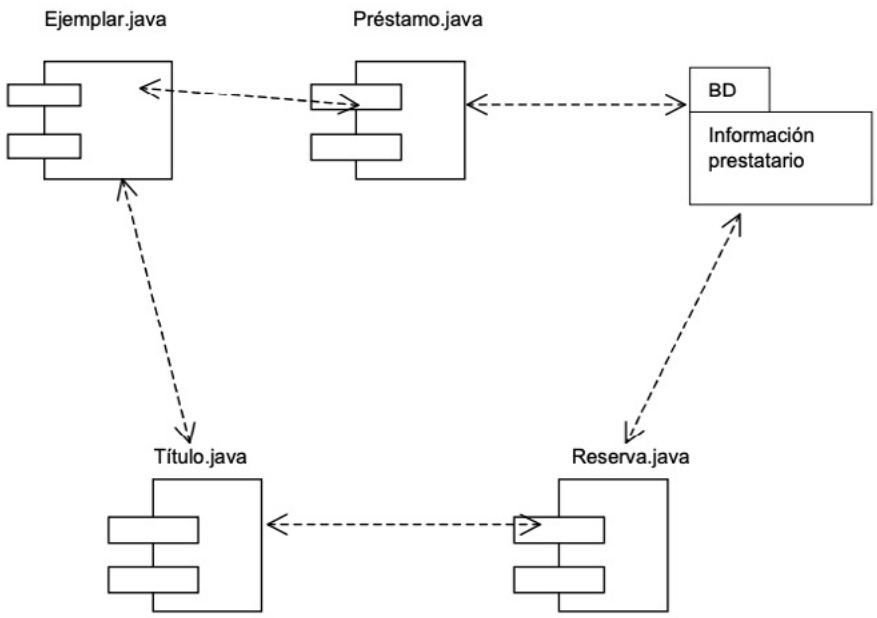
- **Paquetes**: Un paquete es una agrupación de elementos, bien sea casos de uso, clases o componentes. Los paquetes pueden contener a su vez otros paquetes anidados que en última instancia contendrán alguno de los elementos anteriores.
- **Dependencias entre paquetes:** Existe una dependencia cuando un elemento de un paquete requiere de otro que pertenece a un paquete distinto. Es importante resaltar que las dependencias no son transitivas.



6.1.4. Diagrama de componentes

Muestra los **componentes físicos del sistema** (por ejemplo, módulos, bibliotecas, archivos .jar, servicios web), y las **interfaces** que exponen o requieren. Es especialmente útil con sistemas distribuidos, con sistemas modulares o con sistemas basados en microservicios.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Notación:

- **Componente:** Un componente se representa como un rectángulo, con dos pequeños rectángulos superpuestos perpendicularmente en el lado izquierdo.
- **Interfaz:** Se representa como un pequeño círculo situado junto al componente que lo implementa y unido a él por una línea continua. La interfaz puede tener un nombre que se escribe junto al círculo. Un componente puede proporcionar más de una interfaz.
- **Paquete:** Un paquete se representa con un icono de carpeta.

6.1.5. Diagrama de estructura

Permite modelar la **estructura interna de una clase** o componente, es decir, cómo se compone internamente de partes u objetos conectados. Se usa para describir relaciones internas complejas y colaboraciones entre partes.

Ejemplo: la clase Coche podría estar formada por Motor, Ruedas, SistemaFrenado, etc., con sus interacciones internas claramente representadas.

6.1.6. Diagrama de implementación (o despliegue)

Representa la **distribución física del sistema en hardware**: qué nodos (servidores, dispositivos, máquinas) existen y qué componentes se despliegan en ellos.

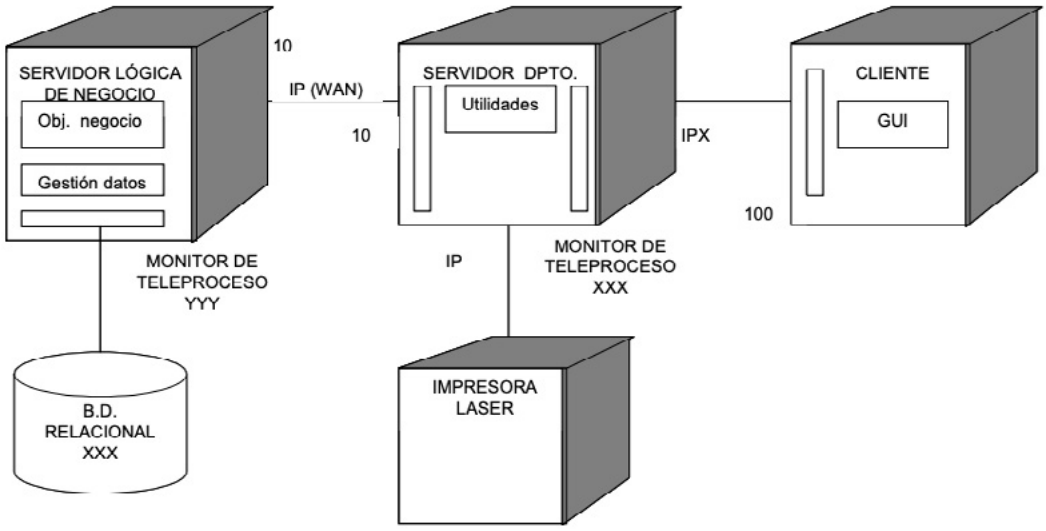
El objetivo de estos diagramas es **mostrar la disposición de las particiones físicas del sistema de información y la asignación de los componentes software a** estas particiones. En estos diagramas se representan dos tipos de elementos, **nodos** y **conexiones.**

- **Nodo**: Se representa con la figura de un cubo. El nodo se etiqueta con un nombre representativo de la partición física que simboliza. Se pueden asociar a los nodos subsistemas de construcción.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Conexión**: Las conexiones se representan con una línea continua que une ambos nodos y pueden tener una etiqueta que indique el tipo de conexión. (ejemplo: canal, red, protocolo, etc.)

Cada nodo se representa como un **rectángulo tridimensional**, y dentro de él se colocan los artefactos desplegados (programas, bases de datos, servicios...).



6.2. DIAGRAMAS DE COMPORTAMIENTO (SECUENCIA, CASOS DE USO, ACTIVIDAD)

Los **diagramas de comportamiento** permiten representar el **comportamiento dinámico del sistema**, es decir, cómo interactúan los objetos, usuarios y procesos a lo largo del tiempo. Son esenciales para analizar, diseñar y documentar la lógica de negocio, los flujos de interacción y los estados de los elementos del sistema.

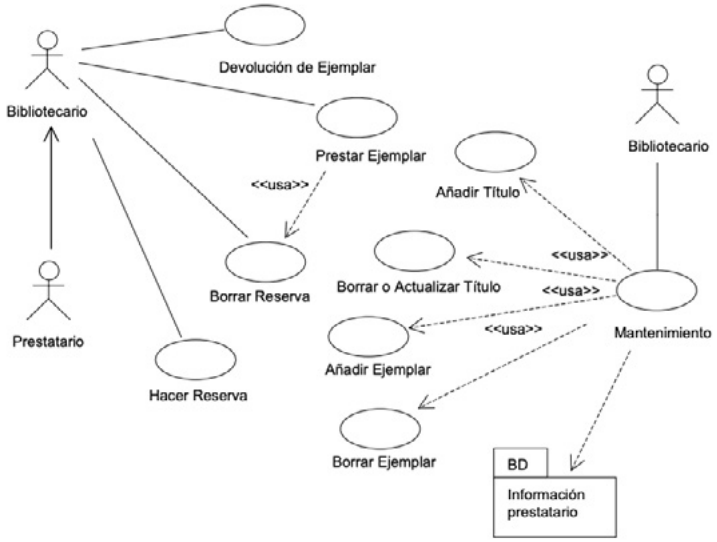
6.2.1.Diagrama de casos de uso

El **diagrama de casos de uso** describe **qué funcionalidades ofrece el sistema a los usuarios o actores externos**. Se centra en el **qué hace el sistema**, no en el cómo. Es especialmente útil en la fase de análisis de requisitos, ya que facilita la comunicación entre técnicos y usuarios.

Elementos principales:

- **Actores**: usuarios u otros sistemas que interactúan con el sistema.
- **Casos de uso**: funcionalidades o servicios ofrecidos por el sistema.
- **Relaciones**: asociaciones, inclusiones, extensiones y generalizaciones entre casos de uso y actores.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



En la siguiente tabla se muestra una descripción de los elementos visuales que se utilizan en la representación de los diagramas de uso:

Elemento	Símbolo / Forma	Descripción
Actor	Figura humana	Representa a un usuario o sistema externo que interactúa con el sistema.
Caso de uso	Elipse u óvalo	Funcionalidad ofrecida por el sistema (ej.: "Iniciar sesión").
Sistema	Rectángulo grande	Caja que agrupa todos los casos de uso que pertenecen al sistema modelado.
Asociación	Línea continua	Conecta un actor con un caso de uso con el que interactúa.
Inclusión	Línea punteada con «include»	Indica que un caso de uso siempre incluye a otro.
Extensión	Línea punteada con «extend»	Un caso de uso opcionalmente amplía otro en ciertas condiciones.
Generalización	Flecha con triángulo vacío	Relación jerárquica entre actores o entre casos de uso.

Dirección de las flechas en las relaciones «include» y «extend»:

- **«include»:** la flecha apunta **desde el caso de uso base hacia el caso de uso incluido**.
- **«extend»:** la flecha apunta **desde el caso de uso de extensión** (el que amplía) **hacia el caso de uso base**.

6.2.2. Diagramas de interacción

6.2.2.1. Diagrama de secuencia

El **diagrama de secuencia** muestra cómo se comunican los objetos entre sí a través de **mensajes en orden temporal**, representando la secuencia de llamadas entre ellos.

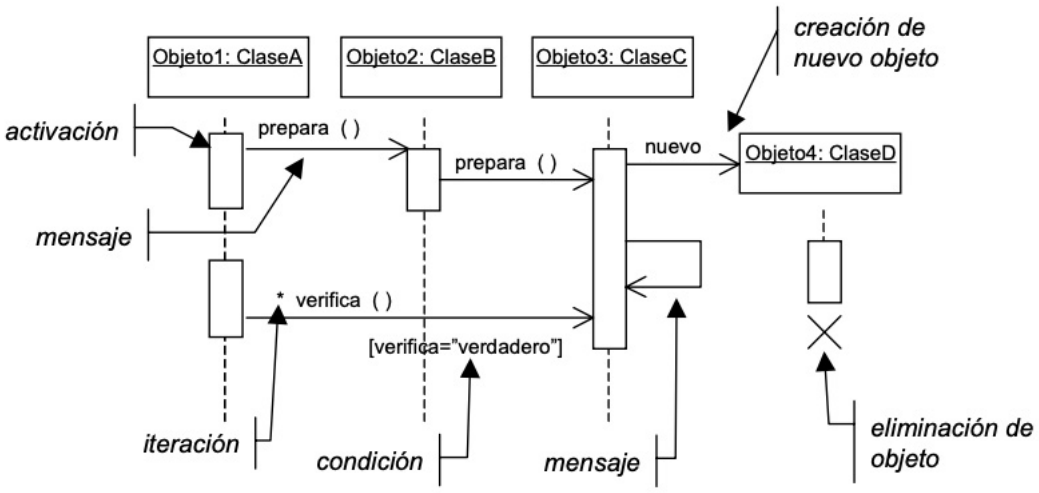
Elementos principales:

- **Participantes u objetos** (en la parte superior).
- **Línea de vida** (vertical) de cada objeto.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Mensajes** que se envían entre objetos.
- **Activaciones** (barras rectangulares que muestran la ejecución de un método).

Es útil para **especificar interacciones complejas** y comprender la lógica de ejecución.



6.2.2.2. Diagrama de colaboración (o de comunicación)

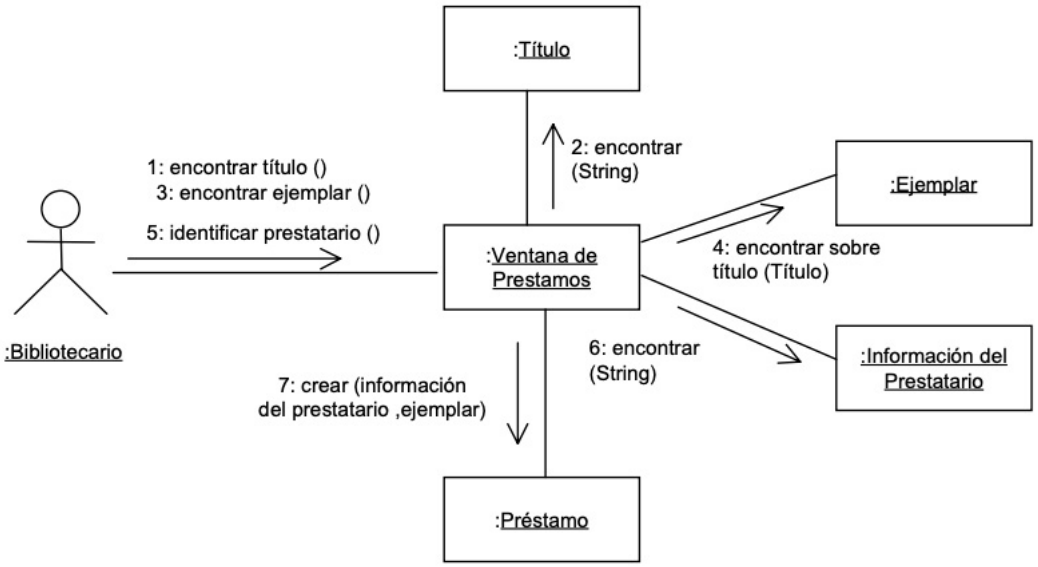
El **diagrama de colaboración** (o **de comunicación**) muestra cómo interactúan los objetos en una situación específica, **resaltando las relaciones entre ellos**. A diferencia del diagrama de secuencia, que enfatiza el tiempo, este enfatiza la estructura de colaboración.

Elementos:

- Objetos conectados mediante enlaces.
- Mensajes numerados que indican el orden de ejecución.

Se utiliza para **entender cómo se distribuye la responsabilidad del comportamiento entre los objetos**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



6.2.3. Diagrama de actividad

El **diagrama de actividad** representa **flujos de trabajo** o procesos dentro del sistema. Es similar a un diagrama de flujo, pero con una semántica más rica que permite modelar decisiones, concurrencia, sincronización y condiciones. Es útil para **modelar lógica de negocio**, flujos de procesos o algoritmos detallados.

6.2.4. Diagrama de estados (máquina de estados)

El **diagrama de estados** representa los **distintos estados** por los que puede pasar un objeto a lo largo de su ciclo de vida, y las **transiciones** que ocurren entre ellos en respuesta a eventos. Muy útil para modelar **componentes con comportamiento reactivo**, como botones, procesos, dispositivos o interfaces.

6.2.5. Diagrama de tiempo (temporal)

El **diagrama de tiempo** muestra cómo **cambian los valores de los atributos de los objetos o sistemas con el paso del tiempo**. Se usa en sistemas de tiempo real o en aquellos donde importa la sincronización precisa.

Elementos:

- Ejes temporales horizontales.
- Estados o valores a lo largo del tiempo.
- Líneas de vida de objetos.

6.2.6. Diagramas de flujo de datos (DFD)

Aunque no son parte de UML, los **DFD (Data Flow Diagrams)** son comunes en análisis estructurado y muy utilizados en la administración pública y entornos legacy. Representan **cómo fluye la información dentro del sistema**, qué procesos la transforman, qué entidades la originan o consumen y qué almacenes la guardan.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Los **Diagramas de Flujo de Datos (DFD)** son herramientas gráficas utilizadas para **representar el flujo de información** dentro de un sistema de información. Se emplean especialmente durante la fase de análisis para comprender cómo se mueven los datos entre procesos, usuarios y almacenamientos.



Un DFD (Data Flow Diagram) representa de forma visual:

- Los **procesos** que transforman datos.
- El **flujo de información** entre procesos, entidades externas y almacenes.
- Las **entradas y salidas** del sistema.

6.2.6.1. Procesos en un DFD

Un **proceso** es una función o actividad del sistema que **transforma entradas en salidas**. Representa **la lógica de negocio**, ya sea automatizada o manual, que se aplica a los datos.

- Cada proceso debe tener **entradas y salidas definidas**.
- **Puede recibir datos de una entidad externa, de otro proceso o de un almacén, y devolver resultados a cualquiera de ellos.**
- **Un proceso no es origen ni final de datos,** se encarga de la transformación o manipulación de los datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipo de proceso	Descripción
Proceso principal	Es el proceso único representado en el diagrama de contexto. Resume todo el sistema.
Proceso de nivel 0	Surge de la descomposición del proceso principal en el primer nivel funcional. Representa funciones de alto nivel.
Proceso intermedio	Es un proceso que ha sido descompuesto en otros procesos más simples, pero que a su vez no es el principal.
Proceso primitivo	Es el último nivel de descomposición, que ya no se divide más. Es la unidad mínima funcional.
Proceso de control (Métrica v3)	En algunos marcos como Métrica v3, se incluyen procesos que coordinan y sincronizan las actividades de otros procesos.



El **DFD de Nivel 0** es una visión global del sistema. Representa el sistema completo como un único proceso principal, junto con sus interacciones con las entidades externas mediante flujos de datos.

- Se le conoce como diagrama de contexto o diagrama 0.
- Incluye una única burbuja o proceso que representa todo el sistema en estudio.
- Muestra las entidades externas que interactúan con el sistema (usuarios, sistemas, organismos...).
- Representa los flujos de entrada y salida entre el sistema y dichas entidades.
- No muestra procesos internos ni almacenes de datos.

6.2.6.2. Flujo de datos

Un **flujo de datos** representa el **movimiento de información** entre procesos, almacenes y entidades externas. Indica qué datos se transfieren, desde dónde y hacia dónde.

- Cada flujo tiene una **dirección única** (una sola flecha).
- Debe representar un **conjunto lógico de datos**, no elementos aislados sin relación.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Tipos:

- **Consulta:** flujo de datos que representa una **lectura** de información desde un almacén de datos o una solicitud de datos desde una entidad externa. Va desde el **almacén** o **proceso que contiene los datos** hacia el proceso o entidad que solicita.

- **Actualización:** flujo que implica una modificación, inserción o eliminación de datos en un almacén de datos. Va desde el proceso que gestiona la operación hacia el almacén de datos.

- **Diálogo:** flujo **bidireccional** que representa una interacción entre el sistema y una entidad externa, como un usuario o sistema externo. Implica **envío y recepción de información**,

6.2.6.3. Almacén de datos

Un **almacén de datos** es un lugar donde los datos se **guardan para su uso posterior**. Puede representar un fichero, base de datos, archivo físico o memoria intermedia.

Tipos:

- **Permanente**: mantiene la información entre sesiones (bases de datos, archivos).
- **Temporal**: usado durante la ejecución del sistema (memoria caché, tablas de trabajo).

Reglas de acceso (lectura/escritura)

- Un almacén **no puede comunicarse directamente** con otro almacén o una entidad externa.
- Siempre debe estar conectado a través de **un proceso** que lea o escriba en él.

6.2.6.4. Entidad externa

Una **entidad externa** es **fuente o destino de datos** para el sistema. Representa a usuarios, otros sistemas o elementos del entorno que **interactúan con el sistema pero están fuera de su control**.

- No ejecutan procesos dentro del sistema, solo **envían o reciben información**.

Representación

- Se representan mediante **una elipse con un nombre en su interior**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- El nombre debe ser **claro, identificable y único** en el sistema.

Los DFD utilizan una simbología estándar para representar cada componente del flujo de datos:

Elemento	Símbolo	Descripción
Proceso	Círculo o rectángulo redondeado	Actividad que transforma datos de entrada en salida.
Almacén de datos	Dos líneas paralelas	Un **almacén de datos** es un lugar donde los datos se **guardan para su uso posterior**. Puede representar un fichero, base de datos, archivo físico o memoria intermedia.
Flujo de datos	Flecha	Indica la dirección y el contenido de los datos que circulan entre elementos.
Entidad externa	Elipse	Fuente o destino externo de los datos (usuarios, sistemas, organizaciones).

6.2.6.5. Uso correcto de los flujos



Flujos válidos

Flujo	Descripción
Entidad ↔ Proceso	La entidad externa envía o recibe datos del sistema.
Proceso ↔ Almacén	Un proceso consulta (lee) o actualiza (escribe) datos.
Proceso ↔ Proceso	Se transmite información entre dos funciones internas.
Proceso ↔ Entidad	El sistema envía respuesta o resultado a un agente externo.

Flujos inválidos

Tipo de conexión	Motivo de invalidez
Entidad ↔ Entidad	No hay procesamiento, el sistema no interviene.
Almacén ↔ Almacén	No hay transformación ni lógica de negocio entre almacenes.
Entidad ↔ Almacén	Las entidades no acceden directamente a los datos del sistema.
Flujo bidireccional sin sentido	Si los datos de ida y vuelta no están claramente definidos.

6.3. MECANISMOS DE EXTENSIÓN EN UML

El **Lenguaje de Modelado Unificado (UML)** proporciona mecanismos que permiten **personalizar y ampliar la notación estándar**, sin modificar su semántica básica. Estos se denominan **mecanismos de extensibilidad**, y su objetivo es adaptar UML a dominios específicos o necesidades concretas del modelado.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Los principales mecanismos de extensión en UML son:

1. **Estereotipos (<< >>)**
   Permiten crear **nuevos elementos semánticos** a partir de los ya existentes. Por ejemplo, se puede definir <<controlador>> o <<DAO>> para clarificar el rol de una clase en una arquitectura.
2. **Valores etiquetados (tagged values)**
   Son **pares clave/valor** que permiten añadir información adicional a cualquier elemento UML. Ejemplo: {versión=1.2} o {estado=experimental}.
3. **Restricciones**
   Son **condiciones o reglas adicionales** que deben cumplirse en el modelo. Se expresan mediante texto libre o en lenguajes formales como OCL (*Object Constraint Language*). Por ejemplo: {cantidad > 0}.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

7. FLUJOGRAMAS O DIAGRAMAS DE FLUJO DE PROCESOS

Los **flujogramas** o **diagramas de flujo de procesos** son representaciones gráficas que describen de manera secuencial los **pasos que componen un proceso**, mostrando cómo se ejecutan actividades, decisiones, entradas y salidas dentro de un sistema, ya sea manual o automatizado.

El modelo **ANSI/SPARC** (propuesto por el American National Standards Institute y el Standards Planning and Requirements Committee en los años 70) establece una **arquitectura de tres niveles** para los sistemas de gestión de bases de datos, con el objetivo de **independizar los datos** del modo en que se almacenan o acceden. Aunque no trata directamente sobre **flujogramas**, sí influye en cómo **se conciben los procesos administrativos y técnicos** desde una perspectiva de bases de datos.

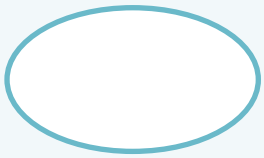
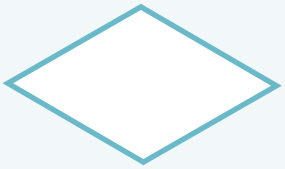
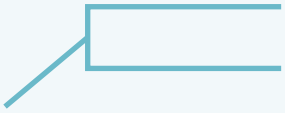
A diferencia de los Diagramas de Flujo de Datos (DFD), que se centran en el movimiento de información, los flujogramas se orientan al **control de flujo de tareas y acciones**.

Diferencias entre DFD y Flujograma

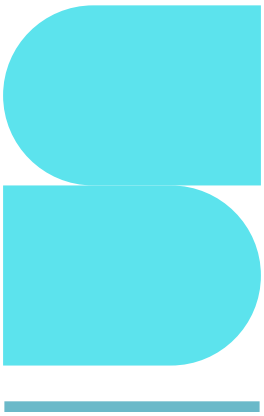
Característica	DFD	Flujograma
Enfoque	Flujo de **datos**	Flujo de **actividades/tareas**
Elemento clave	Procesos, flujos de datos, almacenes	Actividades, decisiones, caminos de ejecución
Nivel	Análisis de sistema / modelado funcional	Procedimientos operativos y ejecución de tareas
Símbolos usados	Círculos, flechas, rectángulos (entidades, almacenes)	Formas normalizadas (óvalos, rombos, rectángulos)
Objetivo	Qué datos se mueven y transforman	Cómo se realizan las tareas paso a paso

Los flujogramas utilizan una simbología estandarizada (basada en ANSI/ISO) para representar los distintos tipos de actividades y decisiones del proceso:

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Nombre	Función	
Inicio / Fin /Terminal	Indica el comienzo o el final del proceso	
Proceso / Tarea	Representa una actividad o paso operativo	
Decisión	Representa una bifurcación (sí / no)	
Conector de flujo	Indican la dirección del proceso	
Conector dentro de la página	Usado para continuar el flujo en otra parte del diagrama	
Entrada/Salida	Indica que el proceso de entrada y salida de datos. Por ejemplo, la visualización de datos.	
Comentario	Amplia la información sobre el nombre de un proceso	
Proceso predefinido	Muestra un proceso que se define en otro lugar.	

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.